HouseVerdi Arithmancy

GenAI project and APL documentation

Claudio Verdi | 24 October 2026

I developed and deployed HouseVerdi Arithmancy on AWS with assistance from AI tools. The application combines programmed numerology calculations with text generated through Amazon Bedrock. On 24 October I verified the web interface, server services, a direct AI request and API validation. The code and setup instructions are available in my public GitHub repository.

Purpose and use

The project is a web application for personal profiles and symbolic comparisons between two people. Users enter names, dates of birth and a gender profile selection. Results cover Pythagorean, Chaldean, Chinese Kua and Indian/Vedic systems, colour and zodiac information. Three role cards provide reflections for partners, friends and colleagues.

Numerology and compatibility percentages are for entertainment and reflection. They are not scientific predictions. AI text can contain errors; one was observed during testing and is documented in this report.

Links and evidence

Web app at verification: http://16.16.184.122/
Repository: https://github.com/HouseVerdi/HouseVerdi_Arithmancy

This report uses my screenshots and terminal output from 24 October, the project files and the supplied Antigravity conversation. The screenshots document the existing solution. They do not show a new complete deployment from an empty server.

Project beginnings in Antigravity

I started in Google Antigravity, where the project foundation took shape. The initial idea compared two people's names for partner, friend and colleague roles. The conversation describes numerology_system.py and an HTML interface. I expanded the requirements with dates of birth, personal mode and additional numerology systems.

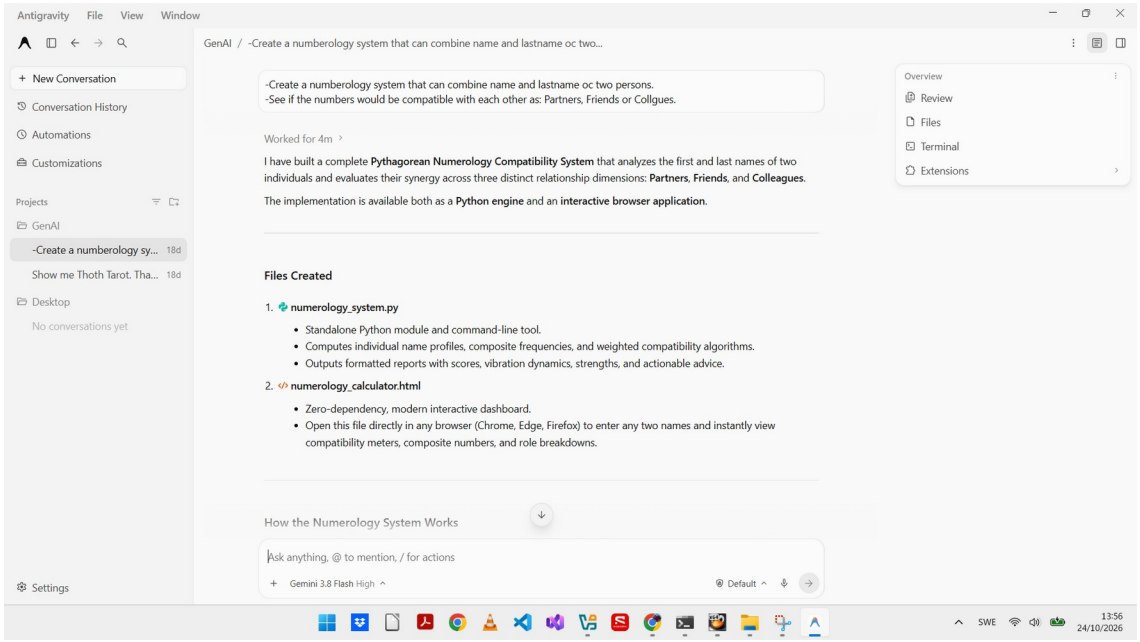


Figure 1. The original project idea in Antigravity

When the usage quota was reached, I continued with ChatGPT. The screenshot shows Individual quota reached on 6 October. Chat descriptions provide history; the later tests show which features actually worked.

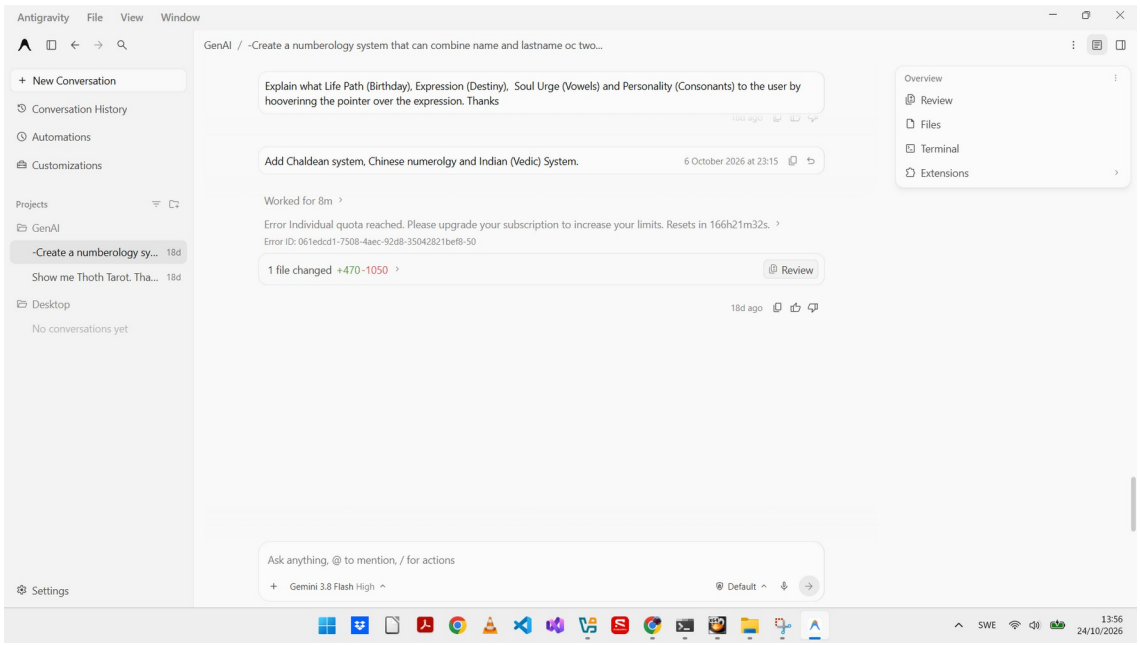


Figure 2. The quota limit that prompted the change of tool

My contribution and project files

I directed the requirements and interface design, tested results and carried out the AWS, troubleshooting and GitHub commands. AI tools assisted with code and instructions. After Antigravity I continued refining the layout, personal default mode, Non-Binary default selection, colours and AI text in the three role cards.

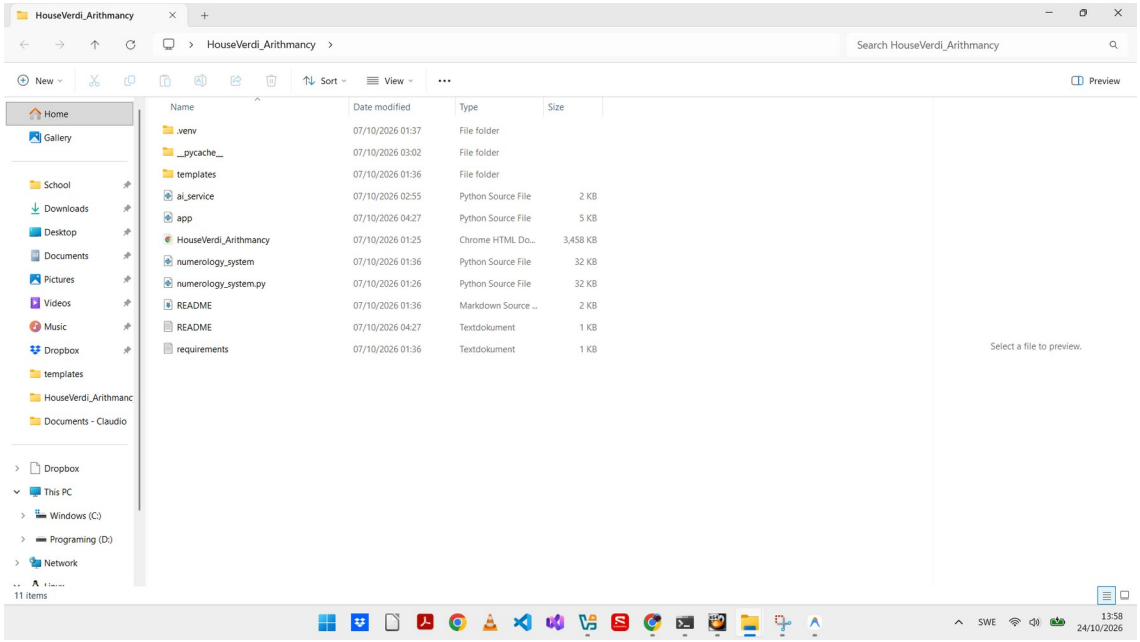


Figure 3. The local project folder in Windows

Files in the published repository

app.py handles Flask routes, validation and AI readings. ai_service.py calls Bedrock. numerology_system.py contains Python calculations. templates/index.html contains the interface, JavaScript calculations and API requests. requirements.txt lists dependencies. README.md describes the project and setup.

The local folder also contained a virtual environment, cache and older files. The published repository uses selected project files and .gitignore. The .venv environment and Python cache are absent from the initial commit.

System architecture

The browser loads the page from Nginx over HTTP on port 80. Nginx forwards requests to Gunicorn at 127.0.0.1:8000. Gunicorn runs Flask. For POST /api/reading, Flask validates input, calculates symbols in Python and calls Amazon Bedrock through boto3. The response contains generated text for partner, friend and colleague roles.

Calculations and generated text

The interface performs its own JavaScript calculations for the visible numbers and scores. The backend also calculates in Python. The browser uses the API response's interpretations for AI text. It is therefore important to distinguish calculated interface results from the model's wording.

In the reviewed code, names, full date of birth, gender and polarity are removed from the person object before symbols are sent to the model. Flask still returns profile information to the user's browser. The model is instructed to return JSON with three text fields. The program checks their presence and displays AI text using textContent.

Configuration and error handling

```
BEDROCK_REGION=eu-north-1
BEDROCK_MODEL_ID=eu.amazon.nova-micro-v1:0
GET /
POST /api/reading
```

ai_service.py uses boto3's credential chain and contains no hardcoded AWS keys. The EC2 screenshots show an IAM role attached to the instance. Its full policy and the runtime credential source were not separately inspected during this verification.

Invalid input produces HTTP 400. Generation failures produce a generic HTTP 502 response in the reviewed code. The interface cancels stale requests when the form changes. HTTP 400 was tested directly; HTTP 502 and concurrency were not load-tested.

AWS instance and operating system

The HouseVerdi-Arithmancy instance has ID i-00f140425a029a738 and public IPv4 16.16.184.122. It runs in Stockholm, eu-north-1, availability zone eu-north-1a. The console showed Running and 3 of 3 status checks passed.

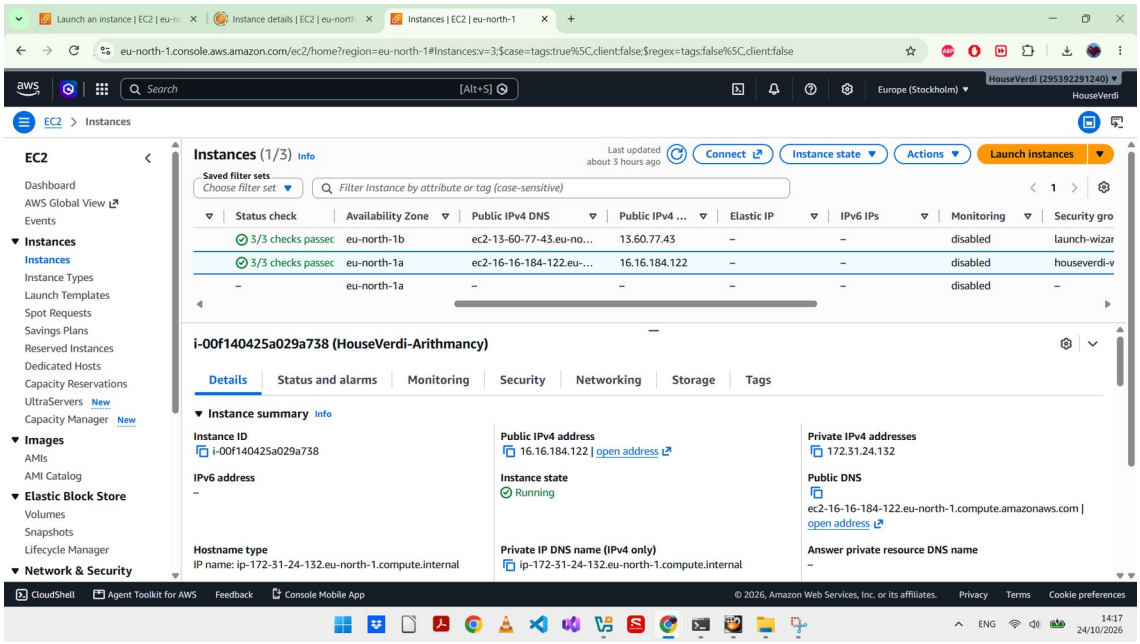


Figure 4. The selected GenAI instance and network addresses

The instance type is t3.micro and its private address is 172.31.24.132. IMDSv2 is Required. The AMI name specifies Ubuntu Noble 24.04 amd64. SSH login on 24 October reported Ubuntu 24.04.5 LTS. The console launch time was 7 October 2026 at 04:09 Swedish summer time.

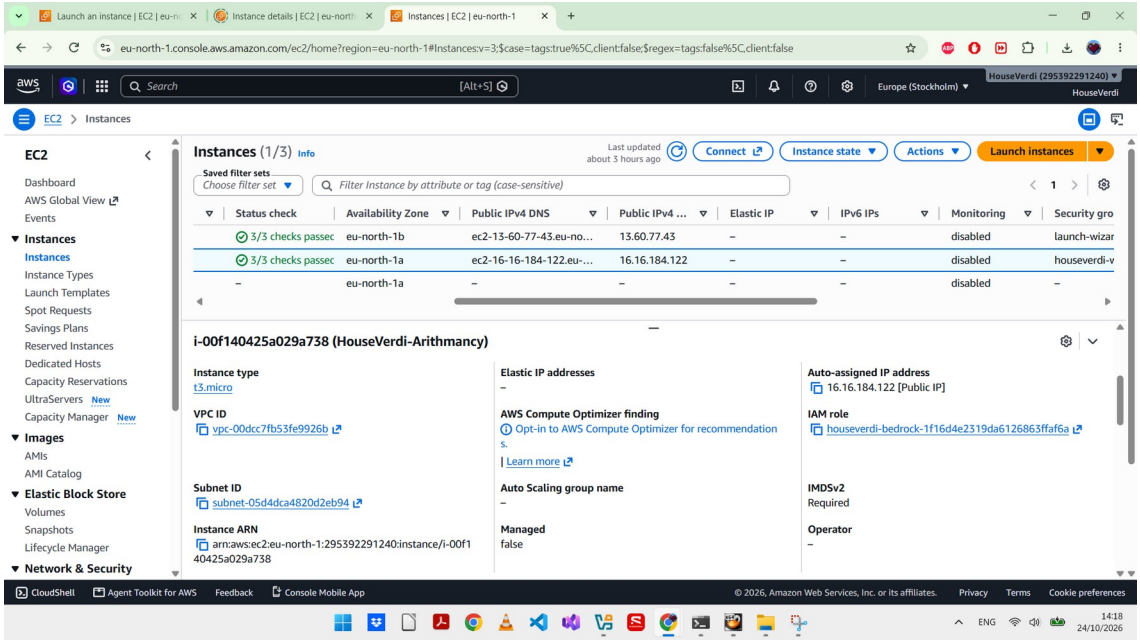


Figure 5. Instance type and attached IAM role

Network access and SSH

The security group permits TCP 22 only from 83.188.29.94/32 and TCP 80 from 0.0.0.0/0. No rule for port 443 is shown in the evidence. The app opened with an explicit http address after an earlier browser timeout.

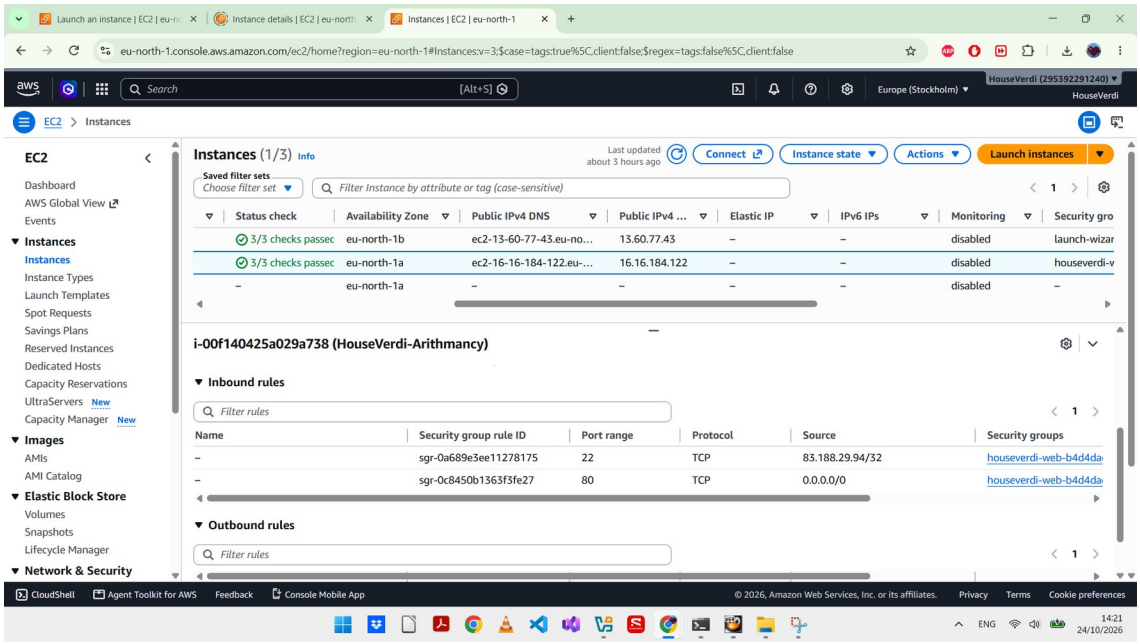


Figure 6. Inbound rules for SSH and HTTP

A search for PEM files did not find the key. In the .ssh folder I found houseverdi_aws without an extension. I connected as ubuntu using this key. The screenshot shows the filename and successful connection, not the key contents.

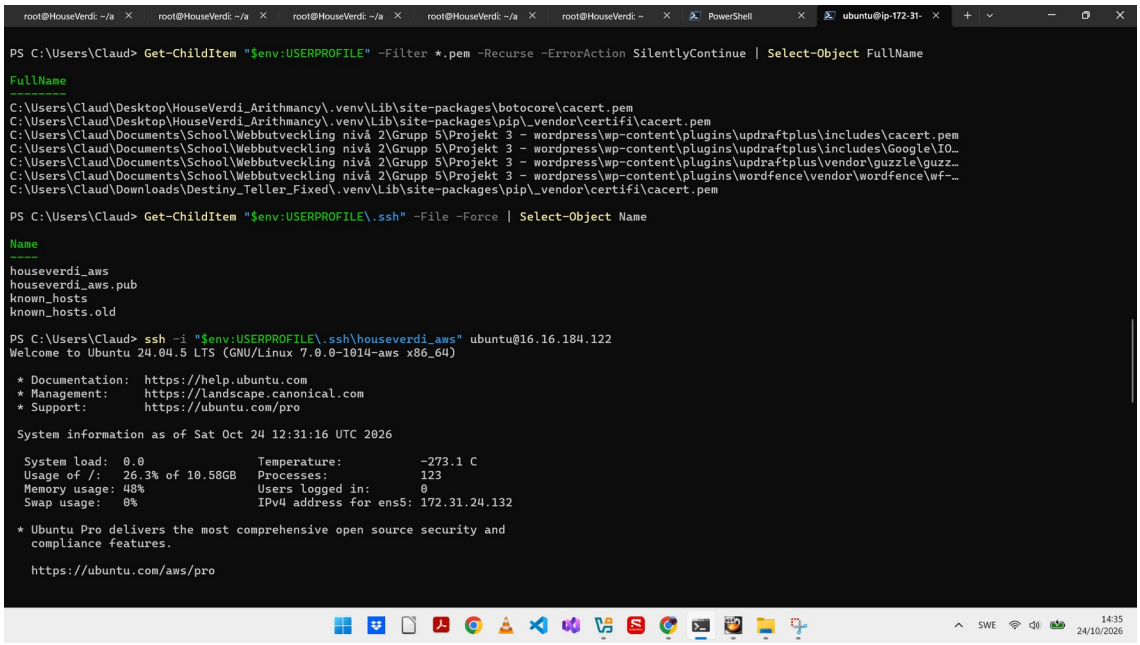


Figure 7. PowerShell connection using the existing SSH key

Services and reverse proxy

systemctl status showed houseverdi and nginx as active running and enabled. Gunicorn 23.0.0 ran with two workers and a 120-second timeout. The log shows binding to 127.0.0.1:8000.

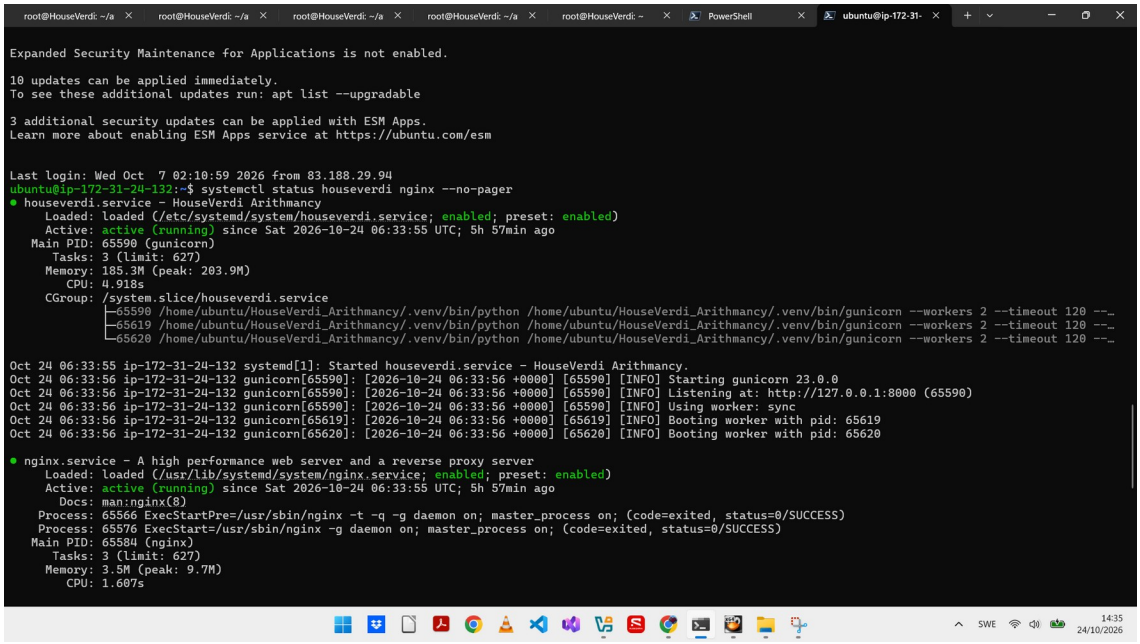


Figure 8. Active systemd services and Gunicorn processes

curl -I against 127.0.0.1 returned HTTP 200 OK from nginx/1.24.0. nginx -T reported a successful configuration test and proxy_pass to the app. The same terminal verification showed the Bedrock region and model and a successful direct AI request.

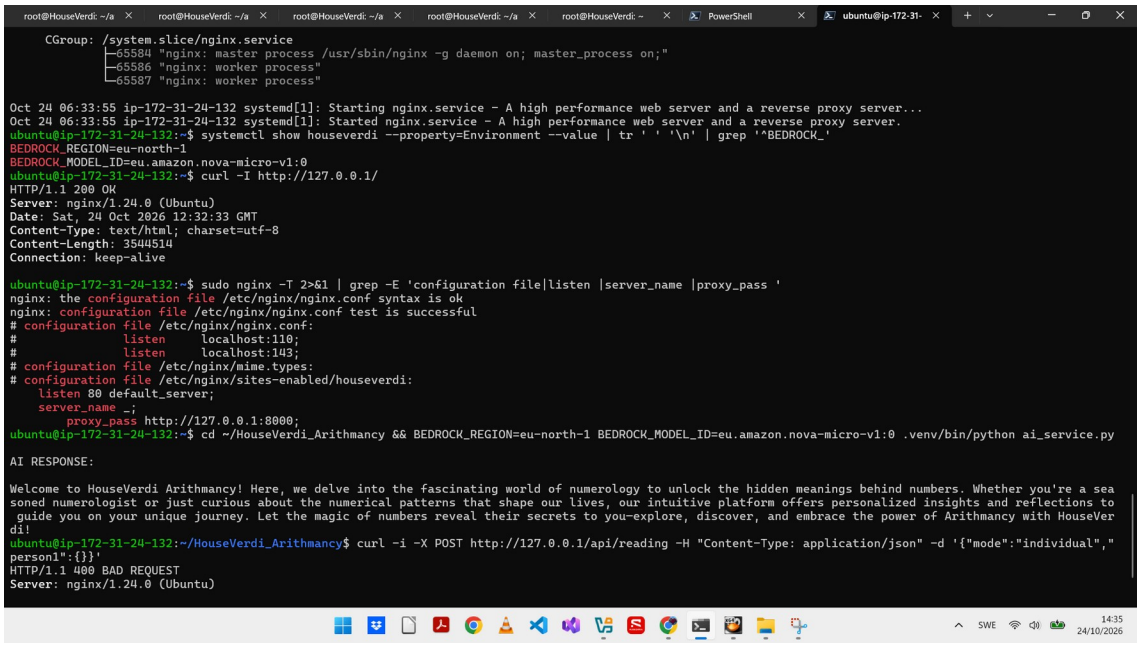


Figure 9. Bedrock configuration and HTTP and Nginx checks

Personal profile test

I used the test profile John Doe, Male, born 15 May 1988. The interface displayed Taurus, Life Path 1, Expression 8, Soul Urge 8 and Personality 9. The colour result was Ruby red with the number 1.

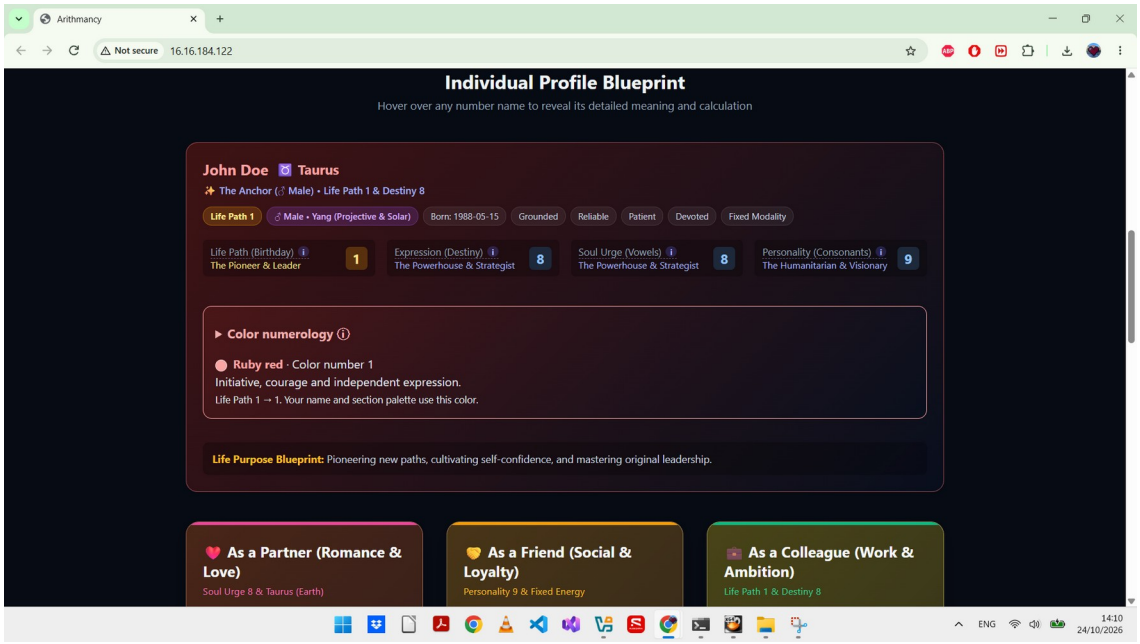


Figure 10. Calculated results for the personal profile

All three role cards displayed AI generated guidance. A content error was observed in the partner text: it referred to Soul Urge and Personality as 9 although Soul Urge was 8. Generation worked, but the text was therefore not fully numerically accurate.

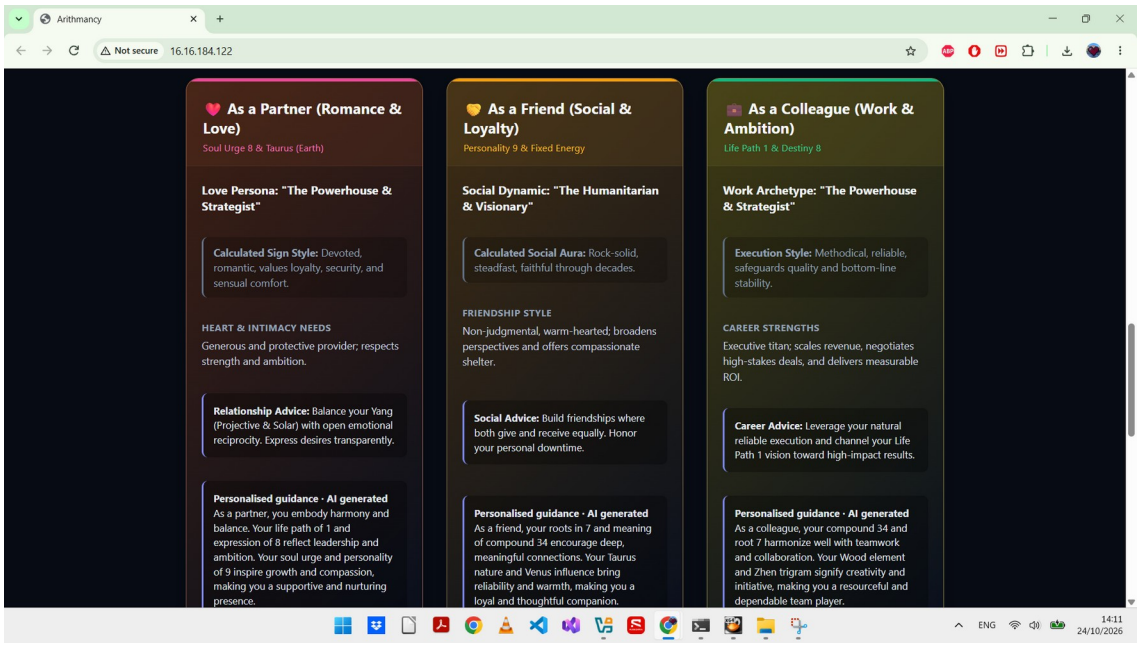


Figure 11. Personal profile AI text including the observed numerical error

Two profile test

I compared John Doe, Male, 15 May 1988 with Jane Dame, Female, 25 April 1990. Jane's profile showed Life Path 3, Expression 8, Soul Urge 3 and Personality 5. These are demonstration data.

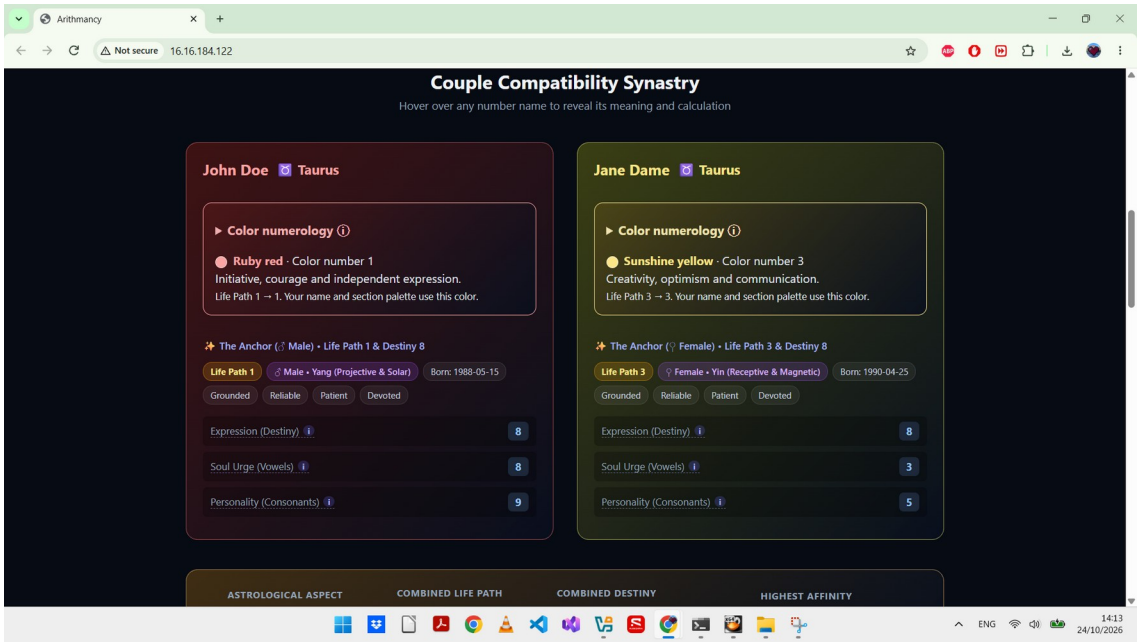


Figure 12. Inputs and profiles in Couple Compatibility

The interface role scores were partner 87 percent, friend 90 percent and colleague 92 percent. Combined Life Path was 4 and Combined Destiny was 7. The percentages come from the application's symbolic formulas and do not represent measured probabilities.

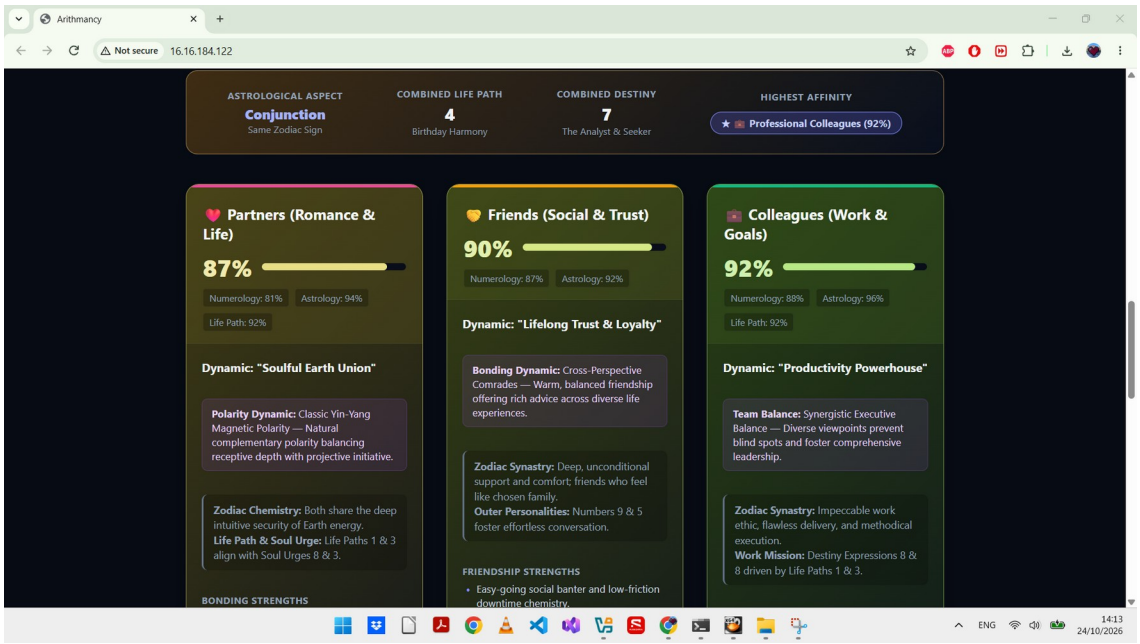


Figure 13. Role scores and combined numbers

Results and AI for two people

The system comparisons displayed Chaldean 90, Chinese Kua 74 and Indian/Vedic 92. These are application calculation results. Kua uses a calendar-year approximation and is not a full BaZi chart.

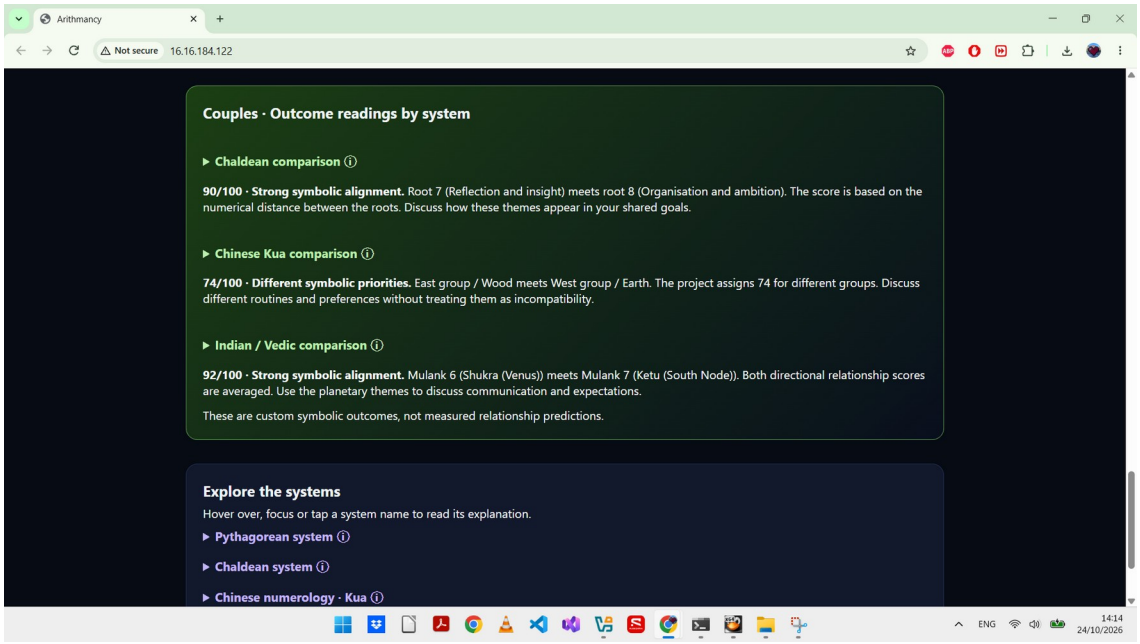


Figure 14. System results in comparison mode

After the AI request completed, generated reflections appeared in all three role cards in couple mode as well. This documents that the interface API flow provided text in both modes. No separate scientific validation of the reflections was performed.

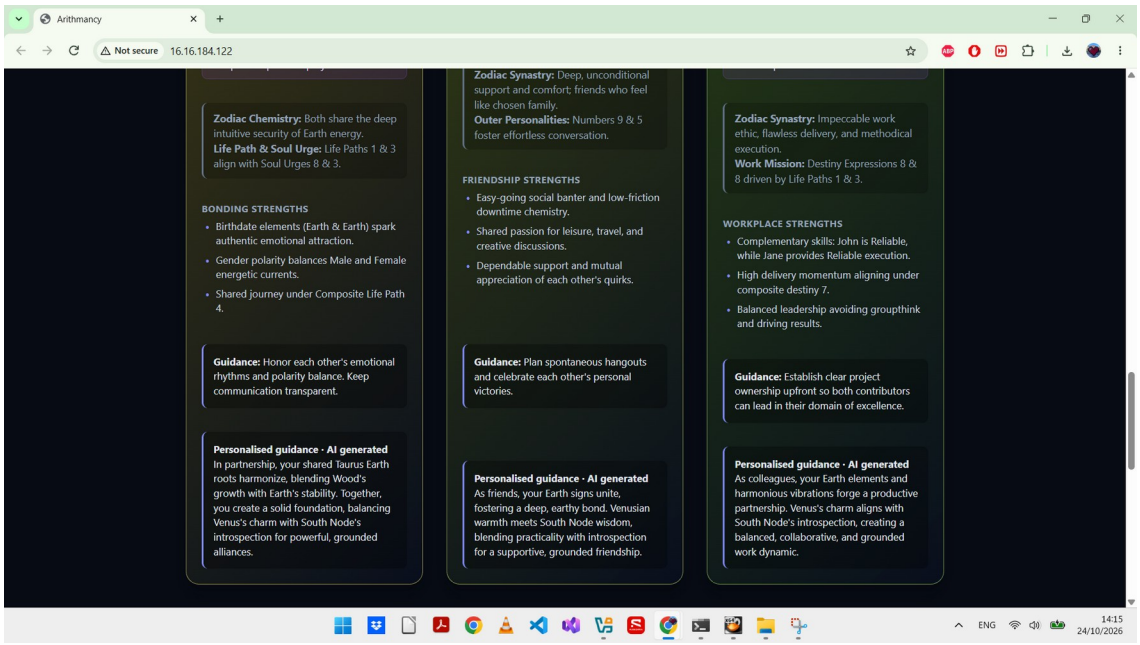


Figure 15. AI-generated text for partner friend and colleague

Direct AI request and API validation

I ran ai_service.py in the server virtual environment with BEDROCK_REGION and BEDROCK_MODEL_ID supplied. The terminal displayed AI RESPONSE and a generated welcome message. This demonstrates a working integration from EC2 at test time.

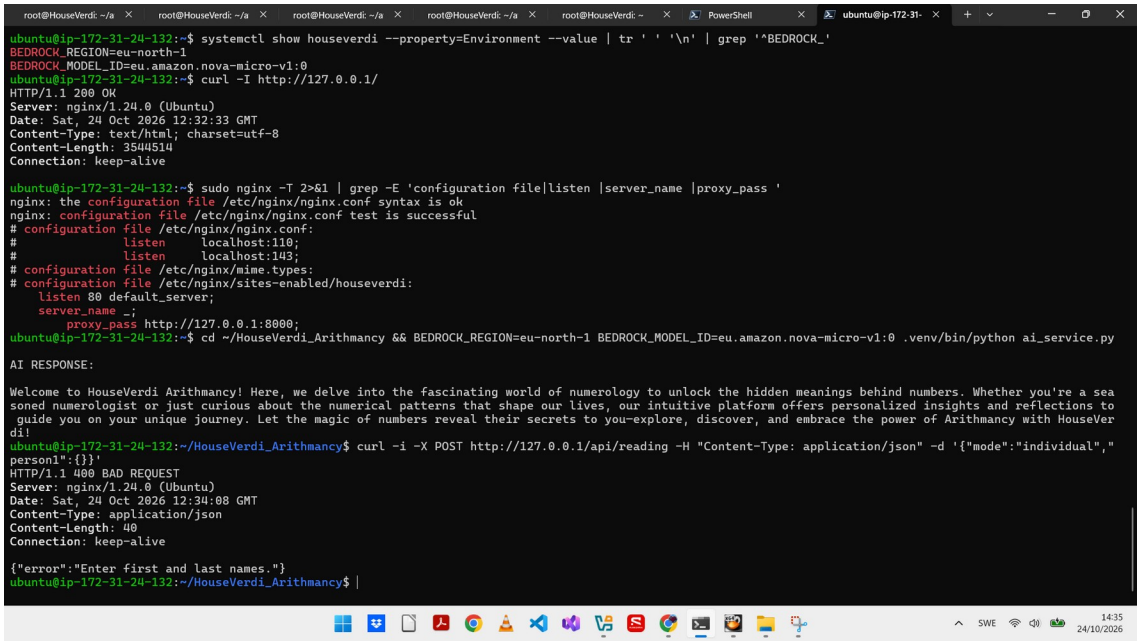


Figure 16. Successful AI request and rejected incomplete API request

I then sent a JSON request with mode individual and an empty person1 object to /api/reading. The response was HTTP 400 BAD REQUEST with Enter first and last names. The test confirms that name validation rejects this incomplete request.

```
curl -i -X POST http://127.0.0.1/api/reading \
  -H "Content-Type: application/json" \
  -d '{"mode":"individual","person1":{}}'
```

The source also checks dates, future birth dates, profile selections and the AI response's three text fields. These checks were reviewed in code and are not additional directly executed tests in this report. No complete automated test suite or load test is reported.

GitHub and version control

I created a public repository under my personal HouseVerdi account. The EC2 project folder initially had no .git directory. I added .gitignore, initialized main and selected six project files. The Git identity was set to Claudio Verdi with the account's noreply address.

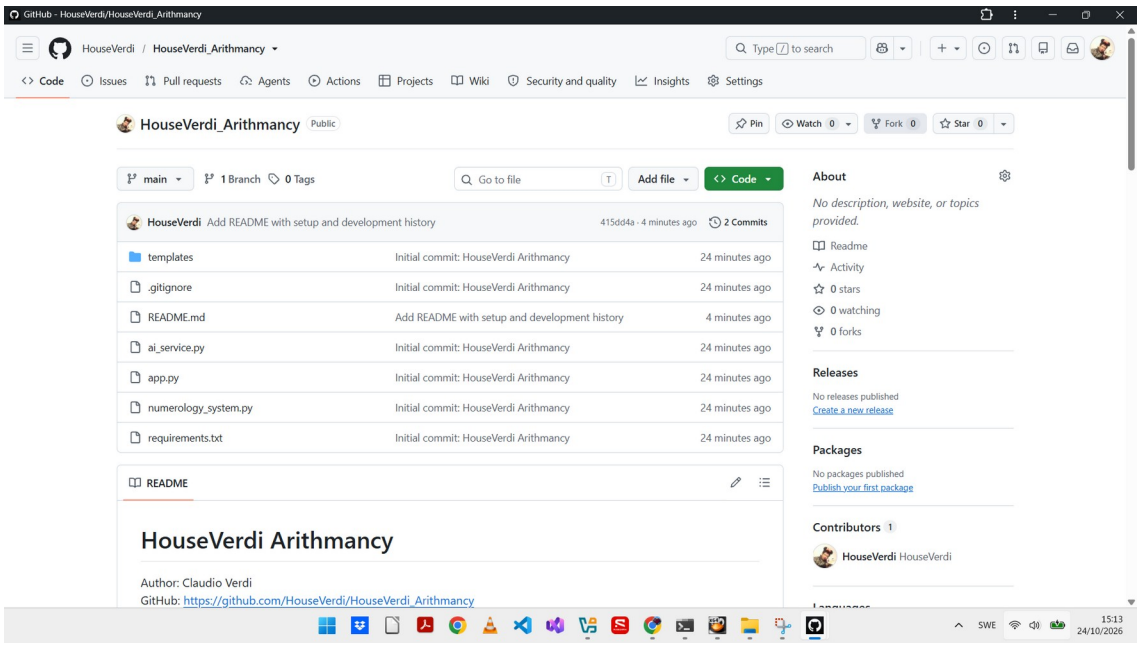


Figure 17. Public repository with two commits and README

Initial commit 3374b75 contains the code. Commit 415dd4a adds the README with setup, history, verification and limitations. I authenticated with GitHub CLI and uploaded using git push. No login codes or tokens are reproduced here.

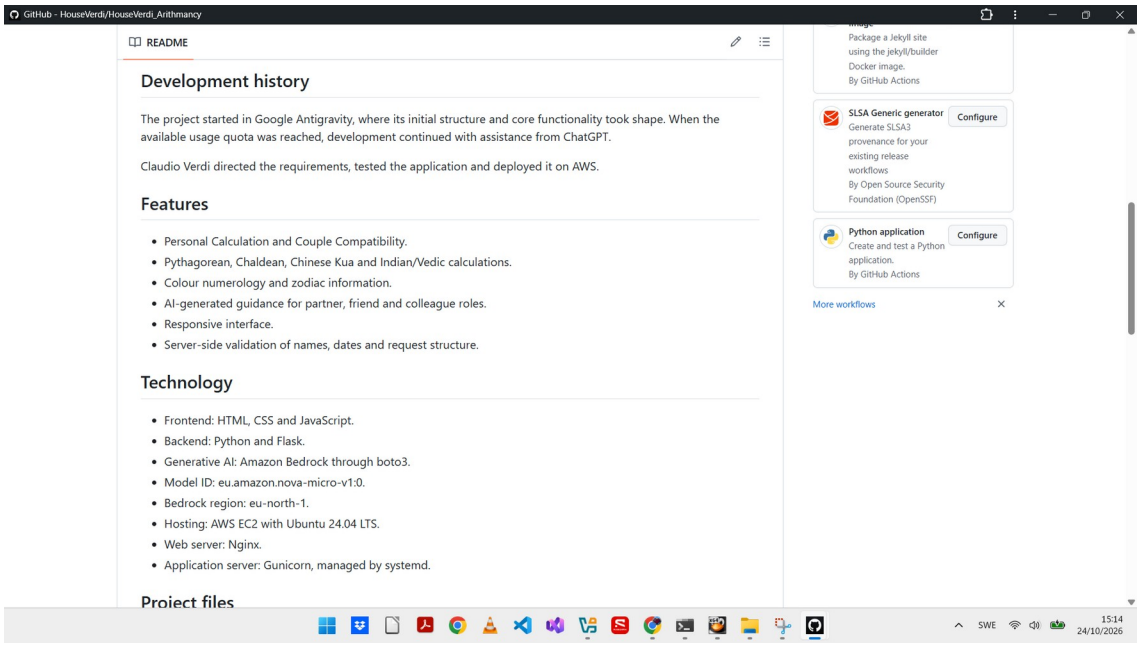


Figure 18. Published development history and technology description

Setup and operation

The README describes local Linux setup using Python, venv and Git. Dependencies are installed from requirements.txt. AWS authentication is configured separately for boto3 and must permit invocation of the selected Bedrock model. Keys must not be stored in the repository.

```
git clone https://github.com/HouseVerdi/HouseVerdi_Arithmancy.git
cd HouseVerdi_Arithmancy
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
export BEDROCK_REGION=eu-north-1
export BEDROCK_MODEL_ID=eu.amazon.nova-micro-v1:0
python app.py
```

The local development server opens at http://127.0.0.1:5000. The existing EC2 deployment instead uses Gunicorn on port 8000 behind Nginx. Opening the HTML file directly from disk is not equivalent to running Flask and its API.

Commands for verified operation

```
systemctl status houseverdi nginx --no-pager
curl -I http://127.0.0.1/
sudo nginx -T
systemctl show houseverdi --property=Environment --value
```

These commands refer to the already installed service. The repository does not contain complete automated provisioning for the server, IAM role, Nginx and systemd. A fully reproducible deployment guide remains an improvement area.

Evaluation and video presentation

The project demonstrates a working GenAI feature in an AWS-hosted web app. I can follow the flow from form through Flask to Bedrock and back to the interface. I have also published the code with a README explaining how the project began and how to run it.

Limitations and improvements

The observed Soul Urge error shows that prompting alone does not ensure accurate numerical claims. I should check model numbers against the calculations or avoid having the model repeat numbers. I should also consolidate JavaScript and Python calculations into one source because separate implementations can produce different scores.

The current Kua calculation uses calendar years without Li Chun adjustment. Non-Binary follows the female Kua formula in the implementation. This simplification should be clear to users. HTTPS, authentication and rate limiting remain improvements. AWS requests incur costs, and no verified cost monitoring is documented here.

Video presentation

I reported that GenAI_Claudio_Verdi.mp4 was recorded and that I checked sound and picture. The recording followed a walkthrough of personal mode, couple mode, service status, model configuration, direct AI request, API validation and GitHub. The video file has not been reviewed as an attachment to this report, so no independent verification of duration or completeness is reported.

Sources and evidence

Evidence consists of the supplied GenAI project brief, Antigravity conversation and screenshots, the project Python and HTML files, requirements.txt, published README, and my terminal output and screenshots dated 24 October 2026. The previous README described an earlier project phase and was replaced on GitHub with the updated version.